Trabajo Práctico unidad 2 SQL

A) Se tiene la siguiente base de datos relacional:

Documentos (cod_documento, descripción)

Oficinas (cod_oficina, codigo_director, descripcion)

Empleados (cod_empleado, apellido, nombre, fecha_nacimiento,

num_doc, cod_jefe, cod_oficina, cod_documento)

Datos_contratos (cod_empleado, fecha_contrato, cuota, ventas)

Fabricantes (cod_fabricante, razón_social)

Listas (cod_lista, descripción, ganancia)

Productos (cod_producto, descripcion, precio, cantidad_stock,

punto_reposición, cod_fabricante)

Precios (cod_producto, cod_lista, precio)

Clientes (cod_cliente, cod_lista, razón_social)

Pedidos (cod_pedido, fecha_pedido, cod_empleado, cod_cliente)

Detalle_pedidos (cod_pedido, numero_linea, cod_producto,

cantidad)

Resolver las siguientes consultas utilizando sentencias SQL:

• Consultas simples (una sola tabla)

1. Obtener una lista con los nombres de las distintas oficinas de la empresa.

   SELECT descripcion

   FROM Oficinas;

2. Obtener una lista de todos los productos indicando descripción del producto, su precio de costo y su precio de costo IVA incluído (tomar el IVA como 21%).

   SELECT descripcion,

```
    precio AS precio_costo,
    precio * 1.21 AS precio_con_IVA
FROM Productos;
```

3. Obtener una lista indicando para cada empleado apellido, nombre, fecha de cumpleaños y edad.

```
SELECT apellido,  Nombre,  Fecha_nacimiento AS fecha_cumpleanos,
TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE()) AS edad

FROM Empleados;
```

4. Listar todos los empleados que tiene un jefe asignado.

```
SELECT *

FROM Empleados

WHERE cod_jefe IS NOT NULL;
```

5. Listar los empleados de nombre “María” ordenado por apellido.

```
SELECT *

FROM Empleados

WHERE nombre = ‘María’

ORDER BY apellido ASC;
```

6. Listar los clientes cuya razón social comience con “L” ordenado por código de cliente.

```
SELECT *

FROM Pedidos

WHERE MONTH(fecha_pedido) = 3

ORDER BY fecha_pedido ASC;
```

7. Listar toda la información de los pedidos de Marzo ordenado por fecha de pedido.

```
SELECT *
```

```
FROM Pedidos

WHERE MONTH(fecha_pedido) = 3

ORDER BY fecha_pedido ASC;
```

8. Listar las oficinas que no tienen asignado director.

```
SELECT *

FROM Oficinas

WHERE codigo_director IS NULL;
```

9. Listar los 4 productos de menor precio de costo.

```
SELECT *

FROM Productos

ORDER BY precio ASC

LIMIT 4;
```

10. Listar los códigos de empleados de los tres que tengan la mayor cuota.

```
SELECT cod_empleado

FROM Datos_contratos

ORDER BY cuota DESC

LIMIT 3;
```

Consultas multitablas

1. De cada producto listar descripción, razón social del fabricante y stock ordenado por razón social y descripción.

```
SELECT p.descripcion,

    f.razon_social,

    p.cantidad_stock

FROM Productos p
```

```
INNER JOIN Fabricantes f

  ON p.cod_fabricante = f.cod_fabricante

ORDER BY f.razon_social ASC, p.descripcion ASC;
```

2. De cada pedido listar código de pedido, fecha de pedido, apellido del empleado y razón social del cliente.

```
SELECT pe.cod_pedido,
    pe.fecha_pedido,
    e.apellido,
  c.razon_social
FROM Pedidos pe
INNER JOIN Empleados e
  ON pe.cod_empleado = e.cod_empleado
INNER JOIN Clientes c
  ON pe.cod_cliente = c.cod_cliente;
```

3. Listar por cada empleado apellido, cuota asignada, oficina a la que pertenece ordenado en forma descendente por cuota.

```
SELECT e.apellido,

    dc.cuota,

    o.descripcion AS oficina

FROM Empleados e

INNER JOIN Datos_contratos dc

  ON e.cod_empleado = dc.cod_empleado

INNER JOIN Oficinas o

  ON e.cod_oficina = o.cod_oficina

ORDER BY dc.cuota DESC;
```

4. Listar sin repetir la razón social de todos aquellos clientes que hicieron pedidos en Abril.

```
SELECT DISTINCT c.razon_social
```

```
FROM Clientes c

INNER JOIN Pedidos p

  ON c.cod_cliente = p.cod_cliente

WHERE MONTH(p.fecha_pedido) = 4;
```

5. Listar sin repetir los productos que fueron pedidos en Marzo

```
SELECT DISTINCT pr.descripcion

FROM Productos pr

INNER JOIN Detalle_pedidos dp

  ON pr.cod_producto = dp.cod_producto

INNER JOIN Pedidos pe

  ON dp.cod_pedido = pe.cod_pedido

WHERE MONTH(pe.fecha_pedido) = 3;
```

6. Listar aquellos empleados que están contratados por más de 10 años ordenado por cantidad de años en forma descendente.

```
SELECT e.apellido,

    e.nombre,

    dc.fecha_contrato,

    TIMESTAMPDIFF(YEAR, dc.fecha_contrato, CURDATE())

      AS anos_antiguedad

FROM Empleados e

INNER JOIN Datos_contratos dc

   ON e.cod_empleado = dc.cod_empleado

WHERE dc.fecha_contrato < DATE_SUB(CURDATE(), INTERVAL 10 YEAR)
```

ORDER BY anos_antiguedad DESC;

7. Obtener una lista de los clientes mayoristas ordenada por razón social.

   SELECT c.*

   FROM Clientes c

   WHERE c.cod_lista = 2

   ORDER BY c.razon_social ASC;

8. Obtener una lista sin repetir que indique qué productos compró cada cliente, ordenada por razón social y descripción.

   SELECT DISTINCT c.razon_social,

      Pr.descripcion

   FROM Clientes c

   INNER JOIN Pedidos pe

     ON c.cod_cliente = pe.cod_cliente

   INNER JOIN Detalle_pedidos dp

     ON pe.cod_pedido = dp.cod_pedido

   INNER JOIN Productos pr

     ON dp.cod_producto = pr.cod_producto

   ORDER BY c.razon_social ASC, pr.descripcion ASC;

9. Obtener una lista con la descripción de aquellos productos cuyo stock está por debajo del punto de reposición indicando cantidad a comprar y razón social del fabricante ordenada por razón social y descripción.

SELECT p.descripcion,

   p.punto_reposicion – p.cantidad_stock AS cantidad_comprar,

   f.razon_social

```
FROM Productos p
INNER JOIN Fabricantes f
   ON p.cod_fabricante = f.cod_fabricante
WHERE p.cantidad_stock < p.punto_reposicion
ORDER BY f.razon_social ASC, p.descripcion ASC;
```

10. Listar aquellos empleados cuya cuota es menor a 50000 o mayor a 100000.

```
SELECT e.*
FROM Empleados e
INNER JOIN Datos_contratos dc
   ON e.cod_empleado = dc.cod_empleado
WHERE dc.cuota < 50000
  OR dc.cuota > 100000;
```

B) Dadas las siguientes tablas:

CLIENTES (código_cliente, nombre, domicilio, provincia)

PRODUCTOS (código_producto, nombre_producto)

ITEM_VENTAS (número_factura, coódigo_producto, cantidad,

precio)

VENTAS (número_factura, código_cliente, fecha)

Resolver las siguientes consultas SQL:

1. Obtener la cantidad de unidades máxima.

```
SELECT MAX(cantidad) AS cantidad_maxima
FROM ITEM_VENTAS;
```

2. Obtener la cantidad total de unidades vendidas del producto c

```
SELECT SUM(cantidad) AS total_unidades

FROM ITEM_VENTAS

WHERE codigo_producto = 'C';
```

3. Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado de mayor a menor por las cantidades vendidas.

```
SELECT p.nombre_producto,

    SUM(iv.cantidad) AS unidades_vendidas

FROM PRODUCTOS p

INNER JOIN ITEM_VENTAS iv

  ON p.codigo_producto = iv.codigo_producto

GROUP BY p.codigo_producto, p.nombre_producto

ORDER BY unidades_vendidas DESC;
```

4. Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado alfabéticamente por nombre de producto para los productos que vendieron mas de 30 unidades.

```
SELECT p.nombre_producto,

    SUM(iv.cantidad) AS unidades_vendidas

FROM PRODUCTOS p

INNER JOIN ITEM_VENTAS iv

  ON p.codigo_producto = iv.codigo_producto

GROUP BY p.codigo_producto, p.nombre_producto

HAVING SUM(iv.cantidad) > 30

ORDER BY p.nombre_producto ASC;
```

5) Obtener cuantas compras (1 factura = 1 compra) realizó cada cliente indicando el código y nombre del cliente ordenado de mayor a menor

```
SELECT c.codigo_cliente,
    c.nombre,
    COUNT(v.numero_factura) AS cantidad_compras
FROM CLIENTES c
LEFT JOIN VENTAS v
  ON c.codigo_cliente = v.codigo_cliente
GROUP BY c.codigo_cliente, c.nombre
ORDER BY cantidad_compras DESC;
```

6. Promedio de unidades vendidas por producto, indicando el código del producto para el cliente 1

```
SELECT iv.codigo_producto,
   AVG(iv.cantidad) AS promedio_unidades
FROM VENTAS v
INNER JOIN ITEM_VENTAS iv
  ON v.numero_factura = iv.numero_factura
WHERE v.codigo_cliente = 1
GROUP BY iv.codigo_producto;
```